

Guía Comprensiva de Data Science en Python

Matplotlib, Pandas, NumPy, Iteración, Indexación y Simulación Monte Carlo

PARTE I: APUNTES DE VISUALIZACIÓN Y MANIPULACIÓN DE DATOS

1. Visualización de Datos con Matplotlib

La librería `matplotlib.pyplot` (convencionalmente importada como `plt`) es la herramienta fundamental en Python para generar gráficos y visualizaciones de datos.

1.1. Gráfico de Líneas (`plt.plot`)

Ideal para representar evolución o tendencias continuas a lo largo del tiempo.

```
import matplotlib.pyplot as plt

year = [1950, 1970, 1990, 2010]
pop = [2.519, 3.692, 5.263, 6.972]

plt.plot(year, pop)
plt.show()
```

1.2. Gráfico de Dispersión (`plt.scatter`)

Utilizado para evaluar la relación o correlación entre dos variables cuantitativas mediante puntos individuales.

```
import matplotlib.pyplot as plt

year = [1950, 1970, 1990, 2010]
pop = [2.519, 3.692, 5.263, 6.972]

plt.scatter(year, pop)
plt.show()
```

1.3. Histograma (`plt.hist`)

Muestra la distribución de frecuencia de un conjunto de datos numéricos dividiéndolos en intervalos o contenedores (bins).

```
values = [0, 0.6, 1.4, 1.6, 2.2, 2.5, 2.6, 3.2, 3.5, 3.9, 4.2, 6]

# Define la cantidad de intervalos (bins) en los que agrupar
plt.hist(values, bins=3)
plt.show()
```

1.4. Personalización Avanzada de Gráficos

Añadir etiquetas, títulos y personalizar las marcas de los ejes (ticks) permite crear gráficos profesionales y comprensibles.

```
import matplotlib.pyplot as plt

year = [1950, 1951, 1952, ..., 2100]
pop = [2.538, 2.57, 2.62, ..., 10.85]

plt.plot(year, pop)

# Personalización de etiquetas y título
plt.xlabel('Year')
plt.ylabel('Population')
plt.title('World Population Projections')

# Personalización de los ticks del eje Y (valores reales vs. etiquetas visuales)
plt.yticks([0, 2, 4, 6, 8, 10], ['0', '2B', '4B', '6B', '8B', '10B'])
plt.show()
```

2. Estructuras de Datos: Diccionarios en Python

Los diccionarios permiten almacenar pares de clave: valor (key-value), ofreciendo una búsqueda rápida mediante claves únicas en lugar de índices numéricos.

```
# Estructura previa poco eficiente con listas separadas:
pop = [30.55, 2.77, 39.21]
countries = ["afghanistan", "albania", "algeria"]

# Implementación con Diccionario:
world = {"afghanistan": 30.55, "albania": 2.77, "algeria": 39.21}

# Acceso rápido por clave
print(world["albania"]) # Salida: 2.77
```

2.1. Comparativa: Listas vs. Diccionarios

Característica	Lista (list)	Diccionario (dict)
Acceso / Operaciones	Seleccionar, actualizar y eliminar con <code>[]</code> ☒	Seleccionar, actualizar y eliminar con <code>[]</code> ☒
Indexación	Indexado por intervalo de números (0, 1, 2...)	Indexado por claves únicas (strings, ints, etc.)
Caso de uso	Colección ordenada de valores donde importa la posición.	Tabla de búsqueda (lookup table) eficiente mediante claves.

3. Introducción a Pandas y DataFrames

La librería pandas es el estándar para el análisis de datos estructurados en Python. El DataFrame es una tabla bidimensional con etiquetas en filas y columnas.

3.1. Creación de un DataFrame a partir de un Diccionario

Las claves del diccionario se convierten en los nombres de las columnas, y las listas de valores representan los datos columna por columna.

```
import pandas as pd

dict = {
    "country": ["Brazil", "Russia", "India", "China", "South Africa"],
    "capital": ["Brasilia", "Moscow", "New Delhi", "Beijing", "Pretoria"],
    "area": [8.516, 17.10, 3.286, 9.597, 1.221],
    "population": [200.4, 143.5, 1252, 1357, 52.98]
}

brics = pd.DataFrame(dict)
```

3.2. Carga de datos desde un archivo CSV

Para cargar un dataset externo se utiliza `pd.read_csv()`. El parámetro `index_col = 0` especifica que la primera columna del archivo CSV debe usarse como el índice de las filas.

```
brics = pd.read_csv("path/to/brics.csv", index_col=0)
```

4. Indexación y Selección de Datos en DataFrames

4.1. Acceso a Columnas

Para seleccionar columnas manteniendo el formato de DataFrame, se pasa una lista de nombres de columnas dentro de corchetes dobles (`[...]`).

```
# Selecciona múltiples columnas retornando un DataFrame
brics[["country", "capital"]]
```

4.2. Acceso a Filas (Slicing / Rebanado)

Para seleccionar un rango de filas se utiliza la sintaxis de rebanado con corchetes simples `[inicio:fin]`, donde el índice final es exclusivo.

```
# Obtiene las filas desde la posición 1 hasta la 3 (índices 1, 2 y 3)
brics[1:4]
```

Resumen de Selección Rápida:

- `df['columna']`: Retorna una Series de Pandas.
- `df[['columna1', 'columna2']]`: Retorna un DataFrame con las columnas seleccionadas.
- `df[1:4]`: Realiza un rebanado (slicing) por rango de filas.

PARTE II: GUÍA DE INDEXACIÓN Y FILTRADO EN PANDAS Y NUMPY

Resumen visual e interactivo de selección de datos con `loc`, `iloc`, comparadores y operadores booleanos.

1. Selección con loc (Por Etiqueta / Nombre)

El método `loc[...]` se utiliza para acceder a filas y columnas mediante sus etiquetas (índices o nombres de columna).

A. Acceso a filas específicas por su etiqueta de índice

```
brics.loc[["RU", "IN", "CH"]]
```

	country	capital	area	population
RU	Russia	Moscow	17.100	143.5
IN	India	New Delhi	3.286	1252.0
CH	China	Beijing	9.597	1357.0

B. Acceso a todas las filas y columnas específicas

```
brics.loc[:, ["country", "capital"]]
```

	country	capital
BR	Brazil	Brasilia
RU	Russia	Moscow
IN	India	New Delhi
CH	China	Beijing
SA	South Africa	Pretoria

2. Selección con iloc (Por Posición Numérica)

El método `iloc[...]` permite seleccionar elementos según su posición de índice basada en enteros (0-indexed).

Acceso a filas y columnas por posición entera:

```
brics.iloc[[1, 2, 3], [0, 1]]
```

	country	capital
RU	Russia	Moscow
IN	India	New Delhi
CH	China	Beijing

3. Comparadores y Operaciones Vectorizadas en NumPy

Los operadores de comparación se aplican elemento a elemento en arrays de NumPy y DataFrames de Pandas, devolviendo arrays booleanos (True/False).

Comparador	Significado	Ejemplo de Uso
<	Menor que	<i>bmi < 22</i>
<=	Menor o igual que	<i>bmi <= 21.75</i>
>	Mayor que	<i>bmi > 23</i>
>=	Mayor o igual que	<i>bmi >= 24.7</i>
==	Igual a	<i>country == "Brazil"</i>
!=	No igual a / Diferente de	<i>capital != "Moscow"</i>

Filtrado condicional en NumPy

```
import numpy as np

np_height = np.array([1.73, 1.68, 1.71, 1.89, 1.79])
np_weight = np.array([65.4, 59.2, 63.6, 88.4, 68.7])
bmi = np_weight / np_height ** 2

# 1. Crear máscara booleana
bmi > 23
# Devuelve: array([False, False, False,  True, False], dtype=bool)

# 2. Filtrar array usando la máscara booleana
bmi[bmi > 23]
# Devuelve: array([24.747])
```

4. Operadores Booleanos Compuestos

Para combinar múltiples condiciones lógicas en NumPy o Pandas, se utilizan las funciones vectorizadas de NumPy (o los operadores de bits &, |, ~):

- `np.logical_and(cond1, cond2)`: Ambas condiciones deben ser verdaderas.
- `np.logical_or(cond1, cond2)`: Al menos una condición debe ser verdadera.
- `np.logical_not(cond)`: Invierte los valores booleanos (True → False).

Ejemplo: Filtrado compuesto en NumPy

```
np.logical_and(bmi > 21, bmi < 22)
# Devuelve: array([ True, False,  True, False,  True], dtype=bool)
```

5. Filtrado de DataFrames en Pandas mediante Máscara Booleana

Podemos filtrar filas de un DataFrame pasando una serie booleana dentro de los corchetes `df[...]` o mediante `loc`.

A. Filtrado por una sola condición (Superficie > 8 millones)

```
is_huge = brics["area"] > 8
brics[is_huge]
```

	country	capital	area	population
BR	Brazil	Brasilia	8.516	200.4
RU	Russia	Moscow	17.100	143.5
CH	China	Beijing	9.597	1357.0

B. Filtrado condicional compuesto (Superficie entre 8 y 10 millones)

```
import numpy as np

# 1. Generar la serie booleana
np.logical_and(brics["area"] > 8, brics["area"] < 10)
# BR      True
# RU      False
# IN      False
# CH      True
# SA      False
# Name: area, dtype: bool

# 2. Aplicar el filtro al DataFrame
brics[np.logical_and(brics["area"] > 8, brics["area"] < 10)]
```

	country	capital	area	population
BR	Brazil	Brasilia	8.516	200.4
CH	China	Beijing	9.597	1357.0

PARTE III: GUÍA DE ITERACIÓN EN PYTHON

Sintaxis y ejemplos de bucles en Diccionarios, NumPy (2D) y Pandas DataFrames.

1. Iteración en Diccionarios

Para iterar sobre los pares clave-valor de un diccionario en Python se utiliza el método `.items()`.

Sintaxis general:

```
for var in seq:
    expression
```

dictloop.py

```
world = { "afghanistan": 30.55,
          "albania": 2.77,
          "algeria": 39.21 }

for key, value in world.items():
    print(key + " " + str(value))
```

SALIDA POR CONSOLA:

```
algeria 39.21
afghanistan 30.55
albania 2.77
```

2. Matrices de NumPy 2D

En arrays bidimensionales de NumPy, usar un bucle for simple iteraría sobre filas completas. Para recorrer elemento a elemento todos los datos de la matriz se emplea `np.nditer()`.

`nploop.py` Iteración secuencial:

```
import numpy as np

np_height = np.array([1.73, 1.68, 1.71, 1.89, 1.79])
np_weight = np.array([65.4, 59.2, 63.6, 88.4, 68.7])
meas = np.array([np_height, np_weight])

for val in np.nditer(meas):
    print(val)
```

SALIDA POR CONSOLA:

```
1.73
1.68
1.71
1.89
1.79
65.4
59.2
63.6
88.4
68.7
```

3. Iteración en DataFrames con `iterrows()`

El método `.iterrows()` permite recorrer un DataFrame fila por fila, devolviendo en cada paso una tupla con la etiqueta del índice (lab) y el objeto Series de la fila (row).

`dfloop.py`

```
import pandas as pd
brics = pd.read_csv("brics.csv", index_col=0)

for lab, row in brics.iterrows():
    print(lab)
    print(row)
```

SALIDA POR CONSOLA:

```
BR
country      Brazil
capital      Brasilia
area         8.516
population   200.4
Name: BR, dtype: object
RU
country      Russia
capital      Moscow
area         17.1
population   143.5
Name: RU, dtype: object
IN
...
```

4. Impresión Selectiva y Manipulación de Columnas

A) Impresión selectiva

dfloop.py

```
import pandas as pd
brics = pd.read_csv("brics.csv", index_col=0)

for lab, row in brics.iterrows():
    print(lab + ":" + row["capital"])
```

SALIDA POR CONSOLA:

```
BR: Brasilia
RU: Moscow
IN: New Delhi
CH: Beijing
SA: Pretoria
```

B) Agregar columna en bucle

dfloop.py

```
import pandas as pd
brics = pd.read_csv("brics.csv", index_col=0)

for lab, row in brics.iterrows():
    # Creating Series on every iteration
    brics.loc[lab, "name_length"] = len(row["country"])

print(brics)
```

SALIDA POR CONSOLA:

	country	capital	area	population	name_length
BR	Brazil	Brasilia	8.516	200.40	6.0
RU	Russia	Moscow	17.100	143.50	6.0
IN	India	New Delhi	3.286	1252.00	5.0
CH	China	Beijing	9.597	1357.00	5.0
SA	South Africa	Pretoria	1.221	52.98	12.0

5. Optimización y Vectorización con apply()

Añadir columnas iterando con `iterrows()` crea una nueva Serie en cada iteración, ralentizando el código. La alternativa eficiente y profesional es utilizar `.apply()`.

dfloop.py

```
import pandas as pd
brics = pd.read_csv("brics.csv", index_col=0)
brics["name_length"] = brics["country"].apply(len)
print(brics)
```

Recomendación: Prefiere siempre `.apply()` o funciones vectorizadas de Pandas/NumPy en lugar de bucles explícitos sobre DataFrames.

SALIDA POR CONSOLA:

	country	capital	area	population	name_length
BR	Brazil	Brasilia	8.516	200.40	6
RU	Russia	Moscow	17.100	143.50	6
IN	India	New Delhi	3.286	1252.00	5
CH	China	Beijing	9.597	1357.00	5
SA	South Africa	Pretoria	1.221	52.98	12

PARTE IV: SIMULACIÓN Y NÚMEROS ALEATORIOS EN PYTHON

Guía resumen basada en NumPy y Matplotlib: desde la generación pseudoaleatoria hasta simulaciones Monte Carlo e histogramas.

1. Generadores Aleatorios y Semillas (Seed)

Los números aleatorios en computación son pseudoaleatorios: se generan mediante una fórmula matemática determinista a partir de un valor inicial llamado semilla (seed).

```
import numpy as np

# Generación pseudoaleatoria estándar [0.0, 1.0)
np.random.rand() # Resultado variable, ej: 0.9535543896720104

# Establecer semilla para reproducibilidad
np.random.seed(123)
np.random.rand()
# Devuelve: 0.6964691855978616

np.random.rand() # Siguiendo número en la secuencia determinista
# Devuelve: 0.28613933495037946
```

2. Lanzamiento de Moneda (Lógica Básica)

game.py

Utilizando `np.random.randint(a, b)` se generan enteros aleatorios en el intervalo `[a, b)`. Para una moneda, `randint(0, 2)` produce 0 (cara/heads) o 1 (cruz/tails).

```
import numpy as np

np.random.seed(123)
coin = np.random.randint(0, 2) # Genera 0 o 1
print(coin)

if coin == 0:
    print("heads")
else:
    print("tails")
```

SALIDA POR CONSOLA:

```
0
heads
```

3. Secuencia de Experimentos (10 Lanzamientos)

headtails.py

Podemos simular una serie de lanzamientos repetidos acumulando los resultados en una lista mediante un bucle for.

```
import numpy as np

np.random.seed(123)
outcomes = []
for x in range(10):
    coin = np.random.randint(0, 2)
    if coin == 0:
        outcomes.append("heads")
    else:
        outcomes.append("tails")
print(outcomes)
```

SALIDA POR CONSOLA:

```
['heads', 'tails', 'heads', 'heads', 'heads', 'heads', 'heads', 'tails', 'tails', 'heads']
```

4. Paseo Aleatorio (Random Walk)

headtailsrw.py

Un paseo aleatorio es una secuencia de pasos donde cada estado depende del anterior más una variación aleatoria. In este ejemplo, calculamos el número acumulado de cruces (tails) tras 10 tiradas.

```
import numpy as np

np.random.seed(123)
tails = [0]
for x in range(10):
    coin = np.random.randint(0, 2)
    tails.append(tails[x] + coin)
```

5. Simulación Masiva (100 Ejecuciones)

distribution.py

Para analizar el comportamiento estadístico, se repite la experiencia (el paseo aleatorio de 10 pasos) unas 100 veces y se registra el resultado final acumulado en cada iteración.

```
import numpy as np

np.random.seed(123)
final_tails = []
for x in range(100):
    tails = [0]
    for x in range(10):
        coin = np.random.randint(0, 2)
        tails.append(tails[x] + coin)
    final_tails.append(tails[-1])
print(final_tails)
```

SALIDA POR CONSOLA:

```
[3, 6, 4, 5, 4, 5, 3, 5, 4, 6, 6, 8, 6, 4, 7, 5, 7, 4, 3, 3, ..., 4]
```

6. Visualización de la Distribución con Histograma

distribution.py

Mediante `matplotlib.pyplot.hist()` visualizamos la distribución de frecuencias de los resultados obtenidos en las 100 ejecuciones.

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(123)
final_tails = []
for x in range(100):
    tails = [0]
    for x in range(10):
        coin = np.random.randint(0, 2)
        tails.append(tails[x] + coin)
    final_tails.append(tails[-1])

plt.hist(final_tails, bins=10)
plt.show()
```